Sample Capstone Proposals

These three samples show the level of detail we expect, in two pages at most. Each one follows the eight questions in the template. The teams, the apps and the numbers are made up.

Sample 1: FirstPass

Team name: Side Letter

1. Problem

Before a private equity fund decides whether to pursue a company, a junior analyst reads a 60 to 100 page document and writes a two-page summary. This takes most of a day per deal, and most deals are rejected anyway.

2. User

The users are analysts and associates at mid-size private equity funds. The buyer is the partner who runs the deal team, because every hour saved on a rejected deal goes to a live one.

A general chatbot can summarize a PDF, but analysts still have to recheck every number. We think that checking is the real pain, and we will test this with three analysts before Week 5.

3. MVP

An analyst uploads one deal document. FirstPass returns a two-page Word summary with four sections: business, market, financials and risks. Every number and claim links to its source page, and the analyst can ask follow-up questions in a chat next to the summary.

Not in the MVP: finding new deals, tracking news, transcribing calls or monitoring a portfolio.

4. Data

Real deal documents are confidential, so we cannot use them. We will use 20 public S-1 filings from SEC EDGAR instead: they are free, long and cover the same ground as a deal document.

5. Models and APIs

- **Retrieval:** embeddings and a vector database over each filing, split by section.
- **Generation:** a small, cheap model first. We move to a larger one only if our evaluation shows we need to.
- **Cost:** we estimate under $x per document.

6. Evaluation

A correct summary has numbers that match the filing, a working citation for every claim, and the filing's main risks. We will hand-write a test set from the 20 filings: 50 questions with answers and page numbers, plus 10 questions the filing cannot answer.

7. Risks

Risk	Fallback
The model invents a number	A script checks every number against the source text and flags any that do not match
Tables in PDFs parse badly	Start with clean filings and show the source page next to each answer
We run out of API credits before Demo Day	Set a spending cap, cache results and record a backup demo
We cannot reach real analysts	Ask classmates who have worked in finance

Sample 2: Larder

Team name: Second Helping

1. Problem

The kitchen manager at a 120-bed retirement home plans meals around each resident's diet, allergies and medical conditions, then orders a week of food by hand. Ordering too much wastes food, and ordering too little forces last-minute substitutions that can break a resident's diet.

2. User

The user is the kitchen manager. The buyer is the home's administrator, who owns a fixed food budget per resident and pays for lower waste and fewer emergency orders.

Spreadsheets and menu software can hold the menu, but a change to one resident's diet means redoing the plan and the order by hand.

3. MVP

The manager enters resident profiles and current stock. Larder builds a weekly menu for each resident, then turns all the menus into one order list with quantities. A short note in plain English explains each substitution.

Not in the MVP: connecting to suppliers, placing orders automatically, forecasting demand from past weeks or a mobile app.

4. Data

Resident records are private health data, so we cannot use them. We will generate 100 synthetic residents, including messy free-text diet notes, and combine them with a public recipe dataset and USDA FoodData Central for nutrition values.

5. Models and APIs

- **Rules:** plain code enforces allergies and diet limits, because a safety rule should never depend on a model.
- **Optimization:** a small linear program picks order quantities that cover every meal at the lowest surplus.
- **Generation:** a small, cheap model turns diet notes into structured profiles and writes the substitution notes.
- **Cost:** we estimate under $x per home per week.

6. Evaluation

A correct week has no meal that breaks a resident's rules, an order that covers every planned meal, and less surplus than a hand-built order. We will test 10 weekly scenarios with our 100 synthetic residents, including sudden diet changes and missing stock.

Measure	Target
Meals that break an allergy or diet rule	0
Free-text diet notes read correctly	95%
Planned meals covered by the order	100%
Surplus food compared with a hand-built order	20% less

7. Risks

Risk	Fallback
The model misreads a diet note	The manager confirms each parsed profile before it is used, and the rules check every meal
Staff trust a plan that is not safe to serve	Label every plan as a draft and require the manager to approve it
Synthetic data is too clean to be realistic	Write the messiest notes by hand and ask a kitchen manager to review them
We cannot reach a kitchen manager	Ask classmates with family in care homes for an introduction, and call local homes directly

Sample 3: Portage

Team name: Zero Downtime

1. Problem

Moving a data warehouse from Postgres to Snowflake takes a mid-size data team weeks of manual work. Engineers map every schema, move the data and check that nothing was lost. A mistake can reach production reports without anyone noticing.

2. User

The users are data engineers at mid-size companies. The buyer is the head of data, who pays because a migration blocks other projects and means paying for two warehouses at once.

Scripts and vendor tools can copy tables, but engineers still plan the move and check the result by hand.

3. MVP

An engineer connects Portage to a Postgres database with read-only access. Portage profiles the schema and proposes a migration plan. Once the engineer approves it, Portage loads the tables into Snowflake and writes a report comparing source and target.

Not in the MVP: continuous sync, a zero-downtime cutover, stored procedures or tables over 10 million rows.

4. Data

Production databases are private, so we cannot use them. We will use three public sample databases (Pagila, Chinook and a small TPC-H) and a Snowflake trial account. We will add 30 known problems to them, such as wrong types, nulls in key columns and broken foreign keys.

5. Models and APIs

- **Agents:** a planner writes the migration plan and a reviewer tries to find what it missed, for at most three rounds.
- **Why two agents:** we expect the reviewer to catch mistakes a single agent misses. We will compare both setups and drop the reviewer if it does not earn its cost.
- **Execution:** plain Python scripts do the loading and checks. The model writes plans, and it never runs commands directly.

6. Evaluation

A correct migration moves every table, with matching row counts and checksums, and the plan catches the problems we planted before any data moves. We will test all three databases with their 30 planted problems.

Measure	Target
Tables with matching row counts and checksums	100%
Planted problems caught before loading	90%

Measure	Target
Plans approved without engineer edits	70%
Failed runs that leave the source untouched	100%

7. Risks

Risk	Fallback
An agent damages the source database	Use read-only source credentials and a separate target schema, and require approval before any write
The plan maps a column to the wrong type	Run checks after every load and stop on the first failure
The agents loop and burn tokens	Cap the rounds at three and set a spending cap
The live demo fails on Demo Day	Demo on a small database and record a backup run